

Relatório de Simulação e Otimização: *Optimization Mini-Project*

Mestrado em Engenharia Informática (MEI) — Ano Letivo 2025/2026

Francisco Costa - 131224 (50%)

Guilherme Pinho - 131690 (50%)

Docentes: Nuno Lau e Amaro Sousa

Resumo – Este relatório descreve a implementação e análise de métodos de otimização aplicados ao problema de seleção de nós controladores numa rede SDN (*Software Defined Network*) com 200 nós. Foram implementados dois algoritmos metaheurísticos, GRASP e GA, e um método exato baseado em Programação Linear Inteira (ILP), resolvido com o *lpsolve*. Os resultados são comparados em termos de qualidade da solução e tempo de execução.

Palavras chave – GRASP, Algoritmo Genético, ILP, *lpsolve*, SDN, Otimização Combinatória.

I. INTRODUÇÃO

O problema consiste em selecionar $n = 8$ nós de um conjunto de candidatos numa rede SDN com $|N| = 200$ nós e $|A| = 250$ ligações para instalar controladores SDN. O objetivo é minimizar o atraso médio de caminho mais curto de qualquer nó ao seu controlador mais próximo (*avgNS*), garantindo que o atraso máximo nó-controlador não excede $W_{max} = 700$ e que o atraso máximo entre qualquer par de controladores não excede $C_{max} = 500$.

Foram implementados três métodos: GRASP, GA e ILP com *lpsolve* (timeout de 10 minutos). Cada metaheurística foi executada 10 vezes com limite de 60 segundos por run.

II. GRASP

A. Descrição e Código

O GRASP (*Greedy Randomized Adaptive Search Procedure*) itera entre duas fases durante o tempo limite:

- Construção greedy aleatorizada:** Em cada um dos n passos, avalia o custo marginal de adicionar cada candidato restante à solução parcial. Constrói a RCL (*Restricted Candidate List*) com os candidatos cujo custo satisfaz $c(i) \leq c_{min} + \alpha \cdot (c_{max} - c_{min})$, e seleciona aleatoriamente um elemento da RCL.
- Pesquisa local:** A solução construída é melhorada por *hill climbing* de melhor melhoria — em cada iteração, avalia todas as trocas possíveis entre um nó selecionado e um candidato não selecionado, e move para a melhor troca que melhore o valor de fitness. Repete até não existir melhoria.

A melhor solução encontrada ao longo de todas as iterações é guardada. O código central das duas fases é apresentado de seguida.

Código 1

CONSTRUÇÃO GREEDY ALEATORIZADA (GRASP)

[2pt]

```
1 function sNodes = greedyRandomized(G, Candidates, n,
2   alpha)
3   sNodes = [];
4   for step = 1:n
5     remaining = setdiff(Candidates, sNodes);
6     nRem = length(remaining);
7     % avalia o custo de adicionar cada candidato
8     % restante
9     costs = zeros(1, nRem);
10    for i = 1:nRem
11      partial = [sNodes, remaining(i)];
12      [avgNS, ~, ~] = ObjectiveSNSP(G, partial,
13        true, false, false);
14      costs(i) = avgNS;
15    end
16  end
17  % constroi a RCL com os candidatos dentro do
18  % threshold
19  cMin = min(costs);
20  cMax = max(costs);
21  RCL = remaining(costs <= cMin + alpha * (cMax - cMin));
22  % escolhe aleatoriamente um candidato da RCL
23  sNodes = [sNodes, RCL(randi(length(RCL)))];
24 end
```

Código 2

PESQUISA LOCAL DE MELHOR MELHORIA

[2pt]

```
1 function [sNodes, objVal] = sa_hc_def1(G, sNodes,
2   Candidates, Wmax, Cmax)
3   objVal = calculateFitness(G, sNodes, Wmax, Cmax);
4   improved = true;
5   while improved
6     improved = false;
7     % candidatos fora da solucao atual
8     Others = setdiff(Candidates, sNodes);
9     bestVal = objVal;
10    bestNeighbor = sNodes;
11    % explora todos os vizinhos - troca cada no
12    % da solucao por candidato fora
13    for a = sNodes
14      for b = Others
15        Neigh = [setdiff(sNodes, a), b];
16        val = calculateFitness(G, Neigh, Wmax, Cmax);
17        if val < bestVal
18          bestVal = val;
19          bestNeighbor = Neigh;
20        end
21      end
22    end
23    % avanca para o melhor vizinho se houver melhoria
24    if bestVal < objVal
25      sNodes = bestNeighbor;
26      objVal = bestVal;
27      improved = true;
28    end
29 end
```

A função *calculateFitness* calcula o *avgNS* e penaliza violações de W_{max} e C_{max} com um fator de

100× violação, de forma a guiar a pesquisa para soluções factíveis.

B. Seleção de Parâmetros

O parâmetro α controla o equilíbrio entre greedy puro ($\alpha = 0$, sem aleatoriedade) e aleatoriedade total ($\alpha = 1$). Foram testados valores de $\alpha \in \{0.10, 0.30, 0.50, 0.70, 0.90\}$, todos produzindo $avgNS = 275.89$, como mostra a Tabela I.

TABLE I
TESTE DE PARÂMETROS GRASP (1 RUN DE 60S POR VALOR)

α	Iterações	avgNS
0.10	50	275.89
0.30	61	275.89
0.50	55	275.89
0.70	57	275.89
0.90	52	275.89

Selecionou-se $\alpha = 0.30$ por produzir o maior número de iterações dentro do limite de tempo, mantendo a mesma qualidade de solução.

C. Resultados dos 10 Runs

TABLE II
RESULTADOS DO GRASP ($\alpha = 0.30$, LIMITE = 60S)

Run	Iter.	avgNS	maxNS	maxSS
1	52	275.89	694	495
2	53	275.89	694	495
3	49	275.89	694	495
4	50	275.89	694	495
5	48	275.89	694	495
6	49	275.89	694	495
7	51	275.89	694	495
8	50	275.89	694	495
9	47	275.89	694	495
10	51	275.89	694	495
Mín.	—	275.89	—	—
Média	—	275.89	—	—
Máx.	—	275.89	—	—

O GRASP obteve $avgNS = 275.89$ em todos os 10 runs (consistência 10/10), com $maxNS = 694 \leq W_{max}$ e $maxSS = 495 \leq C_{max}$, satisfazendo todas as restrições.

III. ALGORITMO GENÉTICO (GA)

A. Descrição e Código

O GA mantém uma população de P_{size} soluções ordenadas por fitness. Em cada geração:

- Seleção por torneio:** Para cada pai, sorteia-se k índices aleatórios da população e seleciona-se o de menor índice (melhor fitness).
- Crossover:** Forma-se a união dos genes dos dois pais e selecionam-se n genes aleatoriamente.
- Mutação:** Com probabilidade q , substitui-se um gene aleatório por um candidato ausente da solução.
- Elitismo:** Os m melhores indivíduos da geração anterior são preservados na nova população.

Código 3
NÚCLEO DO CICLO GERACIONAL DO GA

[2pt]

```

1 for i = 1:P_size
2     idx1 = randi(P_size, 1, k);
3     parent1 = P(min(idx1), 1:n);
4     idx2 = randi(P_size, 1, k);
5     parent2 = P(min(idx2), 1:n);
6     % crossover - junta os nos de ambos os pais e
7     % escolhe n aleatoriamente
8     combined = union(parent1, parent2);
9     offspring = combined(randperm(length(combined), n));
10    % mutacao - com probabilidade q substitui um no
11    % por outro candidato
12    if rand < q
13        others = setdiff(Candidates, offspring);
14        offspring(randi(n)) = others(randi(length(others)));
15    end
16    % local search - aplicado com probabilidade
17    % ls_prob (memetic algorithm)
18    if rand < ls_prob
19        [offspring, ~] = sa_hc_def1(G, offspring,
20        Candidates, Wmax, Cmax);
21    end
22    P_prime(i, 1:n) = offspring;
23    P_prime(i, n+1) = calculateFitness(G, offspring,
24    Wmax, Cmax);
25 end
26 if toc(t) >= timeLimit, break; end
27 P_prime = sortrows(P_prime, n+1);
28 % elitismo - guarda os m melhores da geracao anterior
29 new_P = [P(1:m, :); P_prime(1:(P_size-m), :)];
30 P = sortrows(new_P, n+1);

```

B. Melhoria: Memetic Algorithm

Como estratégia além da implementação standard, foi implementado um **Memetic Algorithm** — um GA híbrido que combina evolução populacional com intensificação local individual [1]. No GA standard, o filho entra diretamente na população após crossover e mutação. No GA memético, aplica-se adicionalmente a pesquisa local sa_hc_def1 ao filho com probabilidade ls_prob , melhorando-o antes de entrar na população.

Foram testadas três versões para determinar o valor ideal de ls_prob :

TABLE III
COMPARAÇÃO DAS VERSÕES DO GA

Versão	ls_prob	avgNS médio	Consistência
GA standard	0	278.85	2/10
Memético 100%	1.0	275.89	10/10
Memético 20%	0.2	275.89	10/10

O GA standard é rápido (~3850 gerações por run) mas inconsistente, encontrou o melhor valor conhecido ($avgNS = 275.89$) em apenas 2 das 10 execuções. O memético com $ls_prob = 1.0$ é consistente mas gera apenas 2 gerações por run pois a pesquisa local domina o tempo disponível. O memético com $ls_prob = 0.2$ é o equilíbrio ótimo: todas as 10 execuções encontram $avgNS = 275.89$, com 15–28 gerações por run dentro do limite de 60 segundos.

C. Seleção de Parâmetros

Foram testados tamanhos de população de 100 e 150 indivíduos. Embora 150 oferecesse maior diversidade, permitia

executar apenas 4–8 gerações dentro dos 60 segundos. Por esse motivo, não foi testado o valor 200 e selecionou-se $P_{size} = 100$, que permitiu executar 15–28 gerações, mantendo a mesma qualidade de solução.

Os parâmetros finais selecionados são: $P_{size} = 100$, $q = 0.1$, $m = 5$, $k = 3$, $ls_prob = 0.2$.

D. Resultados dos 10 Runs

TABLE IV
RESULTADOS DO GA MEMÉTICO ($P_{size} = 100$, $q = 0.1$, $m = 5$, $k = 3$, $ls_prob = 0.2$, LIMITE = 60s)

Run	Ger.	avgNS	maxNS	maxSS
1	15	275.89	694	495
2	15	275.89	694	495
3	28	275.89	694	495
4	26	275.89	694	495
5	19	275.89	694	495
6	17	275.89	694	495
7	22	275.89	694	495
8	23	275.89	694	495
9	23	275.89	694	495
10	21	275.89	694	495
Mín.	—	275.89	—	—
Média	—	275.89	—	—
Máx.	—	275.89	—	—

IV. MÉTODO EXATO: ILP COM LPSOLVE

A. Modelo ILP

O modelo ILP formula o problema minimizando o atraso total sujeito às restrições de W_{max} e C_{max} . As distâncias de caminho mais curto δ_s^i entre todos os pares de nós são pré-computadas com `distances(G)` em MATLAB. Define-se $N(s) = \{i \in \text{Candidates} : \delta_s^i \leq W_{max}\}$ como o conjunto de controladores candidatos acessíveis a partir de s .

Variáveis: $z_i \in \{0, 1\}$ vale 1 se o nó i é selecionado como controlador; $g_s^i \in \{0, 1\}$ vale 1 se o nó s é servido pelo controlador i .

$$\text{Minimizar} \quad \sum_{s \in N} \sum_{i \in N(s)} \delta_s^i \cdot g_s^i \quad (1)$$

$$\sum_{i \in \text{Cand.}} z_i = n \quad (2)$$

$$\sum_{i \in N(s)} g_s^i = 1, \quad \forall s \in N \quad (3)$$

$$g_s^i \leq z_i, \quad \forall s \in N, i \in N(s) \quad (4)$$

$$z_i + z_j \leq 1, \quad \forall i, j \in \text{Cand.} : \delta_i^j > C_{max} \quad (5)$$

A restrição (2) impõe exatamente n controladores. A restrição (3) atribui cada nó a exatamente um controlador; ao restringir as atribuições a $N(s)$, assegura implicitamente W_{max} . A restrição (4) impede atribuição a um nó não selecionado como controlador. A restrição (5) proíbe a seleção simultânea de dois controladores com distância superior a C_{max} .

B. Código de Geração do Ficheiro LP

O ficheiro `.lp` é gerado em MATLAB e passado ao `lpsolve`. As partes mais relevantes do script são:

Código 4 FUNÇÃO OBJETIVO E RESTRIÇÃO DE CARDINALIDADE

[2pt]

```

1 % Pre-computar distancias
2 D = distances(G);
3
4 fid = fopen('problem.lp', 'wt');
5 % Funcao objetivo: soma delta_s_i * g_s_i para i em N
6 fprintf(fid, 'min:_\n');
7 for s = 1:nNodes
8     for ci = 1:length(Candidates)
9         i = Candidates(ci);
10        if D(s,i) <= Wmax
11            fprintf(fid, '+_%.4f_g_%.d_%.d_\n', D(s,i), s, i);
12        end
13    end
14 end
15 fprintf(fid, '\n');
16
17 % Restricao: exatamente n controladores
18 for ci = 1:length(Candidates)
19     fprintf(fid, '+_z_%.d_\n', Candidates(ci));
20 end
21 fprintf(fid, '=_%d;\n', n);

```

Código 5 RESTRIÇÕES DE ATRIBUIÇÃO E DE PAR DE CONTROLADORES

[2pt]

```

1 % Restricao: cada no s atribuido a 1 controlador em N
2 (s)
3 for s = 1:nNodes
4     for ci = 1:length(Candidates)
5         i = Candidates(ci);
6         if D(s,i) <= Wmax
7             fprintf(fid, '+_g_%.d_%.d_%.d_\n', s, i);
8         end
9     end
10 end
11
12 % Restricao: g_s_i <= z_i
13 for s = 1:nNodes
14     for ci = 1:length(Candidates)
15         i = Candidates(ci);
16         if D(s,i) <= Wmax
17             fprintf(fid, 'g_%.d_%.d_%.d_%.d_<=z_%.d_\n', s, i, i);
18         end
19     end
20 end
21
22 % Restricao Cmax: pares de candidatos com distancia >
23 Cmax
24 for ci = 1:length(Candidates)
25     for cj = ci+1:length(Candidates)
26         i = Candidates(ci); j = Candidates(cj);
27         if D(i,j) > Cmax
28             fprintf(fid, 'z_%.d_%.d_%.d_%.d_<=1;\n', i, j);
29         end
30 end

```

Código 6 DECLARAÇÃO DAS VARIÁVEIS BINÁRIAS

[2pt]

```

1 % Variaveis z_i binarias (apenas candidatos)
2 for ci = 1:length(Candidates)
3     fprintf(fid, 'Bin_z_%.d;\n', Candidates(ci));
4 end
5
6 % Variaveis g_s_i binarias (apenas pares validos com
7 delta <= Wmax)

```

```

7  for s = 1:nNodes
8      for ci = 1:length(Candidates)
9          i = Candidates(ci);
10         if D(s,i) <= Wmax
11             fprintf(fid, 'Bin_g_%d_%d;\n', s, i);
12         end
13     end
14 end
15 fclose(fid);

```

Ao restringir as variáveis g_s^i apenas aos pares com $\delta_s^i \leq W_{max}$, o modelo gerado é mais compacto do que declarar todas as $|N| \times |\text{Candidates}|$ combinações, reduzindo o número de variáveis e restrições.

TABLE V
RESULTADOS DO LPSOLVE (TIMEOUT = 600s)

Métrica	Valor
Valor objetivo (soma delays)	59260
avgNS	296.30
Lower bound (relaxação LP)	38956
avgNS lower bound	194.78
Gap	52.1%
Tempo total	601.65 s
Iterações B&B	1078594
Nós B&B explorados	67016
Solução ótima provada	Não

Os oito nós selecionados pela solução do `lpsolve` (variáveis $z_i = 1$ no separador *Result* do IDE) são: **50, 63, 74, 89, 96, 97, 99, 107**. O valor objetivo reportado é a soma total dos atrasos de todos os nós ao controlador mais próximo; o *avgNS* obtém-se dividindo por $|N| = 200$.

O `lpsolve` foi interrompido pelo timeout. O gap de 52.1% resulta da comparação entre a melhor solução inteira encontrada e o lower bound da relaxação LP: $\frac{296.30 - 194.78}{194.78} \times 100 \approx 52.1\%$. Este gap não representa a distância real ao valor ótimo — o ótimo pode situar-se em qualquer ponto acima do lower bound. O lower bound (*avgNS* = 194.78) garante apenas que a solução ótima se encontra no intervalo [194.78, 296.30]; o melhor valor encontrado pelas metaheurísticas (275.89) é superior ao lower bound mas inferior à solução do ILP interrompido.

V. ANÁLISE COMPARATIVA

TABLE VI
COMPARAÇÃO DOS TRÊS MÉTODOS

Método	avgNS mín.	avgNS média	Tempo
GRASP ($\alpha = 0.30$)	275.89	275.89	60s/run
GA Memético 20%	275.89	275.89	60s/run
ILP (<code>lpsolve</code>)	296.30	—	601.65s

1. **As metaheurísticas superam o método exato interrompido.** Ambas obtiveram *avgNS* = 275.89 consistentemente, enquanto o `lpsolve` encontrou *avgNS* = 296.30 em 10 minutos. A dimensão do problema (11121 variáveis binárias) torna a convergência do Branch-and-Bound impraticável no tempo dado.
2. **O lower bound do ILP delimita o espaço de soluções.** A relaxação LP fornece um lower bound

garantido de *avgNS* = 194.78. O gap de 52.1% resulta de $\frac{296.30 - 194.78}{194.78} \times 100$ e compara a solução inteira do ILP com esse lower bound — não representa a distância real ao valor ótimo. O ótimo pode situar-se em qualquer ponto acima de 194.78, possivelmente próximo de 275.89.

3. **O GRASP e o GA obtêm consistentemente o mesmo valor objetivo.** Ambos encontram *avgNS* = 275.89 em todas as execuções. Caso os conjuntos de oito nós selecionados sejam também iguais, pode concluir-se que convergem para a mesma solução; caso contrário, são soluções distintas com igual qualidade. Em qualquer dos casos, a convergência para o mesmo valor sugere que 275.89 é próximo do melhor valor alcançável pelas metaheurísticas, embora não seja possível garantir otimalidade sem o método exato convergir.
4. **A extensão Memética melhora o GA standard.** O GA standard encontrava o melhor valor conhecido em apenas 2/10 runs (média 278.85). O GA memético com *ls_prob* = 0.2 encontra o melhor valor conhecido em 10/10 runs, combinando a exploração global do GA com a intensificação local do *hill climbing*.
5. Com um único parâmetro $\alpha = 0.30$, o GRASP obtém o mesmo resultado que o GA memético com menos complexidade de configuração.

VI. CONCLUSÕES

O problema de seleção de controladores SDN com 200 nós revelou-se demasiado complexo para o `lpsolve` provar otimalidade em 10 minutos. As metaheurísticas GRASP e GA memético obtiveram soluções de qualidade superior (*avgNS* = 275.89) em apenas 60 segundos, com consistência total (10/10 runs). A extensão memética do GA, que aplica pesquisa local a 20% dos filhos, constitui a estratégia além da implementação standard: apesar de reduzir o número de gerações executadas, melhora a qualidade dos descendentes e permite atingir o melhor valor conhecido em todas as execuções.

REFERENCES

- [1] Amaro de Sousa e Nuno Lau, *Metaheuristic Optimization Methods*, Simulação e Otimização, Mestrado em Engenharia Informática e Mestrado em Robótica e Sistemas Inteligentes, DETI – Universidade de Aveiro, Ano Letivo 2025/2026.